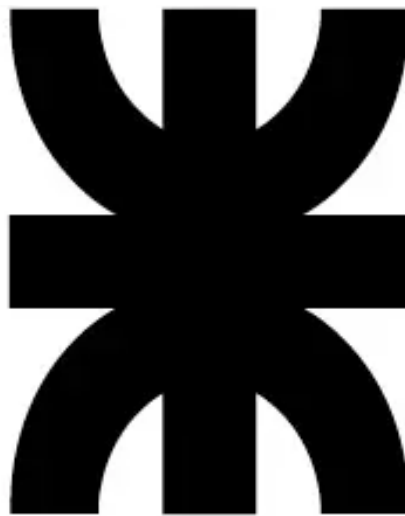


**Universidad Tecnológica Nacional
Facultad Regional Córdoba**

**Ingeniería en Sistemas de Información
Cátedra: Ingeniería y Calidad de Software**

Trabajo Práctico de Test Driven Development (TDD)



Docentes:

- Ing. Judith Meles
- Ing. Cecilia Massano
- Ezequiel Izaguirre
- Marcos Pomenich

Curso: 4K2

Grupo: 7

Alumnos:

- 68719 - Albarracín, Gonzalo Nahuel
- 99395 - Baudo Candela
- 95069 - Hillar, Agustin Nicolás
- 85216 - Koncurat, Joaquin Ernesto
- 96080 - López Román, Rosario
- 94658 - Molina Castillo, Lautaro
- 95035 - Obregón Cáceres, Maximiliano Alexis
- 94262 - Zabalza, Ignacio
- 96612 - Zahr, Luciana Nadín

índice

Introducción	3
Requerimientos técnicos (acordados con el Product Owner)	3
Pruebas y Validación del Sistema mediante TDD (Test Driven Development)	4
Diseño y alcance de las pruebas	4
Implementación técnica	5
Beneficios obtenidos y resultados	5
Backend	6
Frontend	7

Introducción

En este documento se detallarán las decisiones arquitectónicas tomadas como grupo para la implementación de TDD de la User Story “Comprar Entradas” de EcoHarmony Park, junto con los tests utilizados.

Para el desarrollo de este proyecto, hubo una división entre backend y frontend, donde en el backend fue construido en Java script con el framework Node.js, manejando toda la lógica de negocio (comunicación con API de Gmail). El frontend fue realizado con un conjunto de archivos HTML con JavaScript para que se comuniquen con el backend de forma asíncrona y CSS para los estilos. Detallaremos las decisiones arquitectónicas más adelante, primero se explicará cómo se aplicó TDD. Se utilizó Git para el control de versiones y GitHub como repositorio remoto. Se trabajó con ramas por funcionalidad y commits descriptivos, lo que permitió una colaboración ordenada, trazabilidad de cambios y despliegues seguros desde la rama principal.

Requerimientos técnicos (acordados con el Product Owner)

- Realizar una aplicación móvil (simular con web responsive),
- Formato de fecha: DD/MM/AAAA,
- Paleta de colores: [Paleta de colores EcoHarmonyPark](#)
- El tipo de letra usado fue Montserrat,
- Cuerpo de notificación de la compra de entradas efectuada correctamente: llevar a cabo una conexión con gmail donde figuren los datos básicos de la compra (Nombre de la empresa con el resumen de compra), el mensaje de confirmación y código de compra. De forma tal que el visitante recibe:

“EcoHarmony Park - Resumen de compra

Hola <nombre de visitante>!

Gracias por elegir EcoHarmony Park. Aquí tienes el resumen de tu compra:

Código de compra: 5730

Fecha de visita: 2025-10-31

Entradas: <cantidad de entradas compradas>

Detalle:

Visitante 1: edad x, tipo y

Visitante n: edad x, tipo y

Total: \$10000”,

Donde n es el número máximo de visitantes posibles, x es un número que va desde 0 a 99 e y el tipo de entrada (normal o VIP).

- Precio de entradas normales: \$5.000 y las VIPs: \$10.000
- En base a las edades de los visitantes las entradas recibirán el siguiente formato de descuentos:
 - a) Visitantes entre 15 y 60 años un 50% de descuento.

- b) Visitantes de 3 años o menos no pagan entradas.
- Las fechas de visitas son máximo para 1 mes,
- Días en los que el parque se encuentra abierto: martes a domingo.
- No se abren los días 25/12 (navidad), 01/01(año nuevo), ni los lunes.
- Se pueden comprar 10 entradas por transacción como máximo, no hay límite diario.
- Simular conexión con la pasarela de Mercado Pago.

Pruebas y Validación del Sistema mediante TDD (Test Driven Development)

El proceso de desarrollo de este proyecto siguió la filosofía de Desarrollo Guiado por Pruebas (TDD, por sus siglas en inglés), una metodología ampliamente utilizada en entornos de ingeniería de software que busca asegurar la calidad del código mediante la escritura de pruebas antes de la implementación funcional. Este enfoque permite detectar errores tempranamente, reducir el acoplamiento entre módulos y garantizar que cada componente cumpla su propósito de forma predecible.

En el caso particular de este trabajo, se aplicó TDD para validar las principales funciones del frontend relacionadas con la compra de entradas, asegurando la correcta interacción entre los distintos elementos de la interfaz (formularios, validaciones y cálculos de totales). Las pruebas se implementaron utilizando el framework Jest, junto con el entorno jsdom, que emula el comportamiento del DOM en un contexto de pruebas sin necesidad de un navegador real.

Diseño y alcance de las pruebas

Las pruebas unitarias abarcaron la lógica interna del sistema de compra, centrada en la validación de los datos ingresados por el usuario y el correcto funcionamiento de las funciones auxiliares. Para ello, se definieron casos de prueba (CP) que contemplan los distintos escenarios posibles en la compra:

- CP1: Ingreso de cantidad de entradas: se verificó que el sistema rechace valores fuera del rango permitido (por ejemplo, 0 o negativos) y acepte únicamente cantidades válidas, como un máximo de 10 entradas.
- CP2: Selección de fecha de visita: se probaron casos donde la fecha ingresada es inválida, como fechas pasadas, días lunes (cuando el parque permanece cerrado) o fechas excesivamente futuras. Estas pruebas aseguraron que el sistema solo permita seleccionar fechas válidas dentro del rango establecido.

- CP3: Selección de forma de pago: se comprobó que, ante la ausencia de selección de una forma de pago, el sistema identifica el error y no permite continuar con la operación.
- CP4: Validación de la grilla de visitantes: se evaluó que todos los campos correspondientes a los visitantes estén completos, y que se apliquen correctamente los descuentos para menores de 15 años o mayores de 60.

Estas pruebas se enfocaron exclusivamente en el comportamiento de las funciones, no en la comunicación con servicios externos (como el envío de correos electrónicos), priorizando el aislamiento del código y la comprobación unitaria de cada bloque funcional.

Implementación técnica

Cada caso de prueba se implementó utilizando la función `test()` de Jest, dentro de bloques `describe()` que agrupan los distintos conjuntos de validaciones. Antes de cada prueba, se generaba dinámicamente un entorno HTML simulado, representando los formularios y tablas del sistema. Esto permitió interactuar con elementos como `input`, `select` y `radio` sin necesidad de un navegador.

Además, se emplearon `mocks` de funciones y dependencias externas, como `fetch` y `Toastify`, para evitar comportamientos no deterministas y garantizar que cada prueba se mantuviera controlada y reproducible.

```
test('Debe FALLAR si la cantidad es 12', () => {  
  cantidadInput.value = '12';  
  expect(validarCantidadEntradas()).toBe(false);  
});
```

Esta prueba define un escenario concreto, donde la cantidad de entradas excede el límite permitido, y verifica que la función `validarCantidadEntradas()` retorne `false`.

El mismo patrón se replicó para los distintos casos de validación, de modo que cada función fuera verificada bajo condiciones normales y anómalas.

Beneficios obtenidos y resultados

La aplicación de TDD permitió obtener un código más robusto, modular y fácil de mantener. Al definir primero los criterios de éxito mediante pruebas, se obligó a implementar funciones que cumplieran estrictamente los requisitos funcionales.

Durante el proceso, este enfoque también facilitó la detección temprana de inconsistencias lógicas —por ejemplo, en el cálculo de totales o el tratamiento de fechas— que podrían haber pasado inadvertidas en una validación manual.

Asimismo, la estructura de las pruebas favoreció la documentación implícita del comportamiento esperado del sistema, ya que cada caso de prueba describe claramente las condiciones y resultados esperados. Esto resulta especialmente útil en entornos colaborativos, donde distintos miembros del equipo deben comprender rápidamente cómo debe responder cada componente.

Backend

El backend fue desarrollado en **Node.js con Express**. Se construyó una pequeña **API REST** cuyo único propósito es manejar el envío de correos electrónicos a través del **servicio de Gmail**, utilizando **OAuth2** para la autenticación. De esta forma se evita depender de contraseñas directas o configuraciones inseguras de SMTP.

Toda la información sensible del servicio se gestiona a través de un archivo `.env`, donde se almacenan las credenciales del cliente de Google y el token de actualización necesario para enviar correos de forma automatizada.

Para obtener el `refresh token`, se desarrolló un script auxiliar llamado `get-refresh-token.js`. Este script utiliza las credenciales del cliente (`GMAIL_CLIENT_ID`, `GMAIL_CLIENT_SECRET` y `GMAIL_REDIRECT_URI`) definidas en el archivo `.env` para generar una URL de autorización. El usuario debe abrir esa URL, conceder los permisos al servicio, y luego pegar en la consola el código que devuelve Google. Con ese código, el script obtiene los **tokens de acceso y actualización**, mostrando en pantalla el `refresh_token` que luego se copia en el archivo `.env`. Este token permite al servidor renovar el acceso a Gmail sin intervención manual.

El archivo principal del backend es `server.js`, donde se configura el servidor Express, se habilita el uso de CORS y el parseo de JSON con `body-parser`, y se cargan las variables de entorno necesarias mediante `dotenv`. A partir de esas variables se crea un cliente OAuth2 utilizando la librería `googleapis`, con el cual se inicializa la conexión a los servicios de Gmail. El `refresh_token` permite mantener la autenticación activa de forma continua, renovando los tokens de acceso automáticamente cuando es necesario.

El servidor expone un único endpoint: `POST /api/send-email`. Este endpoint recibe desde el frontend o cualquier otro servicio un cuerpo con los campos `subject` y `text`. Con esa información, se genera un mensaje MIME en formato texto mediante una función auxiliar `createMIME`, que construye la estructura del correo y la codifica en base64 para cumplir con los requisitos de la API de Gmail. Luego, se utiliza el método `gmail.users.messages.send` para enviar el

mensaje, donde el remitente (`FROM_EMAIL`) y el destinatario (`TO_EMAIL`) se obtienen del archivo `.env`. En esta versión de la implementación, el destinatario está configurado de forma fija para pruebas, pero puede reemplazarse fácilmente por un valor dinámico, por ejemplo, el correo del usuario que realizó la compra.

Antes de procesar el envío, el servidor valida que los campos `subject` y `text` estén presentes; si falta alguno, devuelve una respuesta de error. Si ocurre un fallo durante la conexión o el envío, se captura la excepción y se devuelve un mensaje genérico al cliente, registrando los detalles del error en la consola del backend para diagnóstico.

El flujo general de uso del servicio consiste en obtener inicialmente el `refresh_token` con el script, agregarlo al `.env`, iniciar el servidor (`node server.js`) y luego realizar la solicitud `POST /api/send-email` desde el cliente o el frontend. Si todo está correctamente configurado, el correo se envía a través de la API de Gmail de manera segura y autenticada.

Frontend

El frontend se implementó como un sitio estático. La estructura de la interfaz se define en un único documento HTML que incluye los estilos propios, Bootstrap por CDN para grillas y utilidades, y Toastify para notificaciones. El JavaScript se organiza en un módulo ES (`scripts.js`) cargado con `type="module"` y un import map que resuelve la librería `random` desde ESM.

La página principal contiene el formulario de compra con tres bloques: selección de fecha, grilla dinámica de visitantes y forma de pago. La fecha de visita se valida totalmente en el cliente: el input se inicializa con `min` hoy y `max` un mes adelante, y se aplica una validación en tiempo real que impide fechas pasadas, fechas con más de un mes de anticipación, lunes (el parque permanece cerrado) y las fechas especiales 25/12 y 01/01. Para mejorar la UX, se anula el feedback nativo del navegador y se reemplaza por bordes de estado (rojo/verde) y toasts centrados arriba. También se bloquean teclas no válidas en el campo de fecha y se evita el uso de datalists o elementos extra; se aprovecha el calendario nativo del navegador.

La sección de cantidad de entradas habilita o deshabilita el botón “Generar grilla” según el valor ingresado (1 a 10). Al generar, la grilla de visitantes se arma por DOM en un `<table>`: por cada visitante se crea una fila con campos de edad (con límite 1–99 y validación inmediata) y tipo de entrada (General/VIP), además de un botón para eliminar la fila. Cada cambio en edad o tipo recalcula el total. La lógica de precios está encapsulada en una función que recorre la grilla actual: General \$5000, VIP \$10000, `<= 15` o `>= 60` aplica 50% de descuento, y `<= 3` años no paga. El total se refleja en tiempo real en el resumen.

En el envío del formulario se centraliza toda la validación: fecha válida, cantidad dentro de rango, datos completos de todos los visitantes y selección de forma de pago. Si se elige tarjeta, el botón de compra muestra un estado “Pagando...” con un breve delay simulado antes de continuar. Una vez superadas las validaciones, se construye un resumen de compra (incluye fecha, cantidad, detalle por visitante y total) y se envía al backend con fetch vía **POST** a <http://localhost:3000/api/send-email>. El cuerpo incluye subject y text en JSON y, si la respuesta es satisfactoria, se muestran toasts de éxito, se limpia el formulario, se oculta el encabezado de la grilla y se reinicia el total. Para el código de compra se utiliza la librería random (importada como módulo ESM), manteniendo la app sin dependencias de build.

El estilo visual se resolvió con una hoja **styles.css** que define una paleta de verdes mediante variables CSS, tipografía Montserrat por Google Fonts, una navbar fija con sombra y un formulario central con tarjeta, bordes redondeados y sombras suaves. Se incluyeron reglas responsive para inputs y botones y una personalización de Toastify (bordes, sombras y centrado real independiente del layout) para mantener coherencia estética con el resto del sitio. Este enfoque evita preprocesadores y herramientas de build, y permite desplegar el frontend como archivos estáticos, integrándose en forma directa con la API del backend para el envío de correos y la confirmación de la compra.